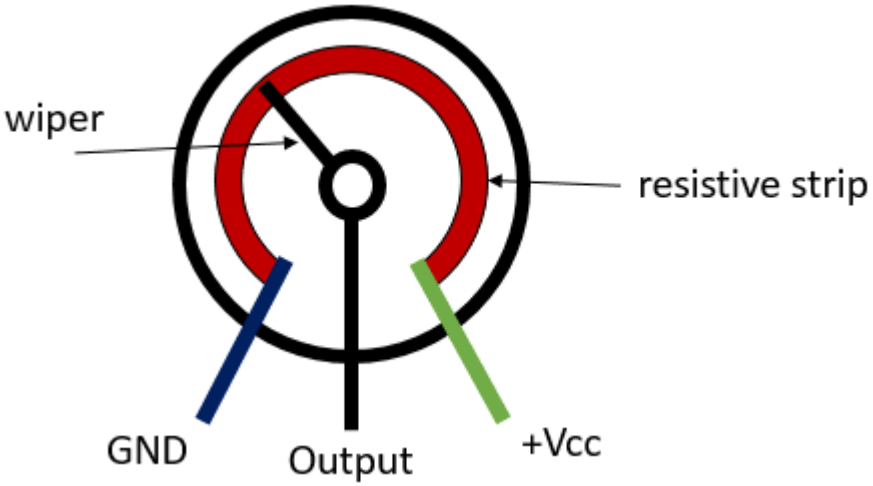
Msc(DS&AI,CS)
IOT and Application

Teaching Scheme: Credit 01
Theory/Practical : (01 hrs./2 hrs.) / week

Examination Scheme:
Practical (PR): 50 Marks
Oral (OR): 50 Markss

Dr. Minaxi Doorwar
Assistant Professor in E&Tc Department
JSPM's ICOER, Wagholi, Pune



A potentiometer is a three-terminal resistor that acts as an adjustable voltage divider by moving a slider (wiper) over a resistive element. It works on the principle that potential drop across a uniform resistive wire is proportional to its length (l), allowing for precise voltage control, measurement of EMF, or internal resistance.

Components: A resistive track (with two end terminals) and a moving contact called a wiper.

Principle (V=k* l) The voltage between one end and the wiper depends on the position of the wiper along the resistance. A uniform current through the wire ensures the voltage drops proportionally to the length.

Voltage Divider: By connecting a voltage source across the outer terminals, the center terminal (wiper) provides an adjustable output voltage from 0V to the full input voltage.

Types:

Slider/Slide pot: Linear movement.

Rotary pot: Rotational movement.

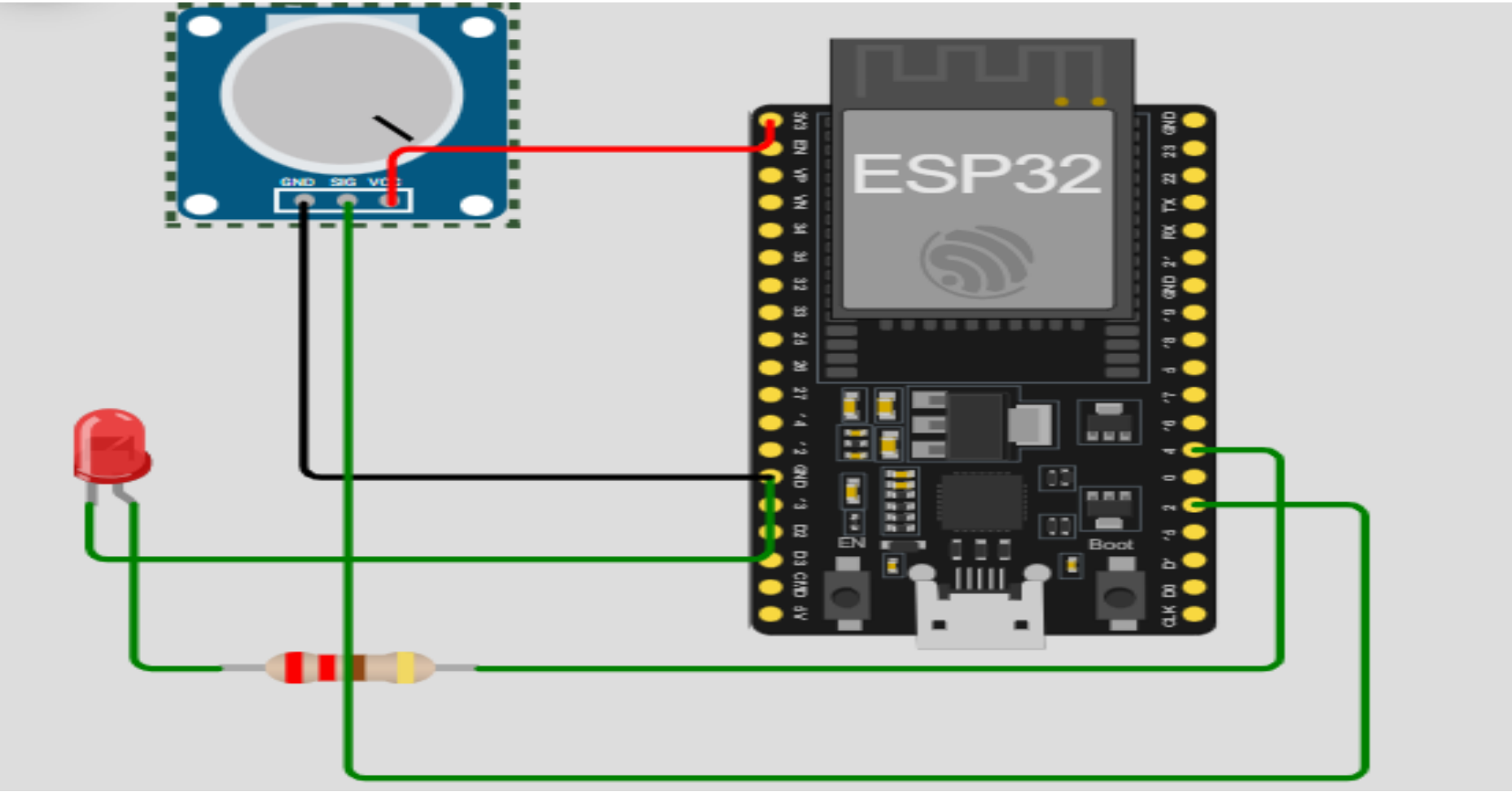
Trimmer pot: Small, used for one-time or infrequent calibration.

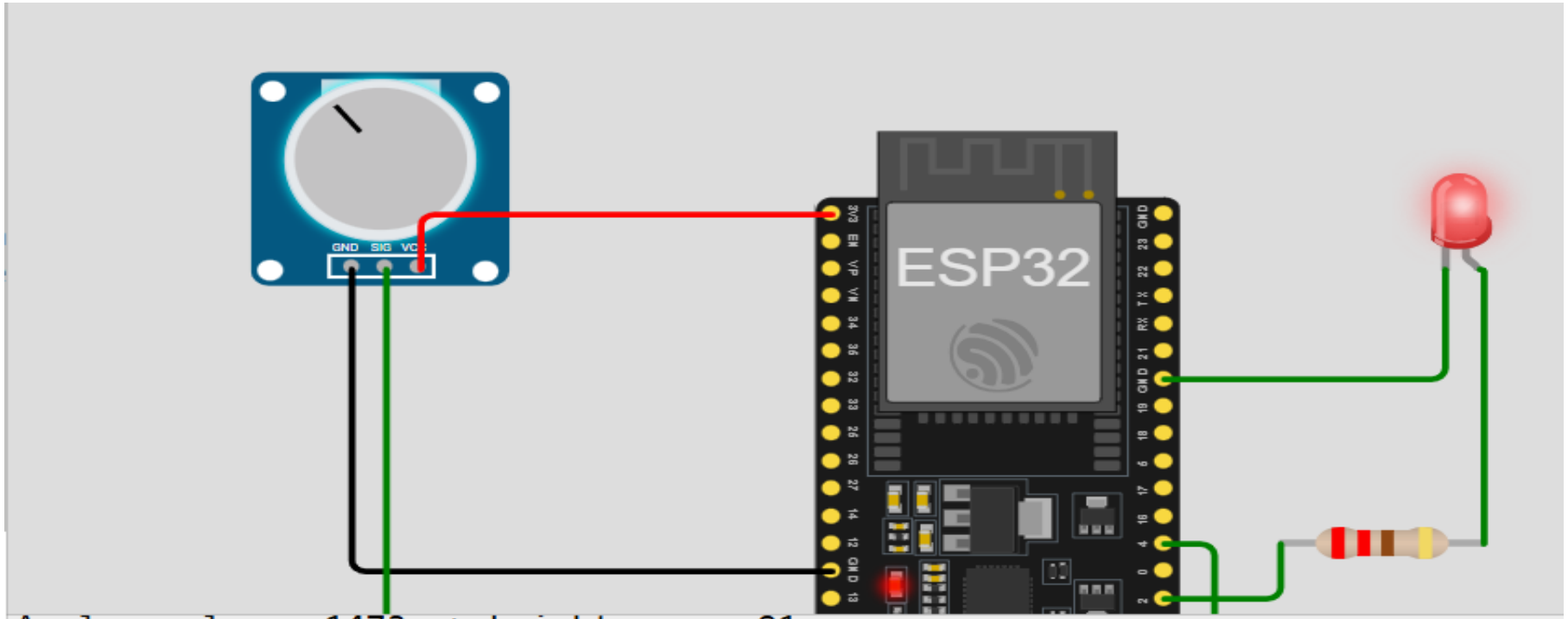
Applications: Volume control, joystick sensors, tuning, and calibrating electrical instruments

Control the brightness by using a potentiometer

```
#define POTENTIOMETER_PIN  2                    // ESP32 pin connected to Potentiometer pin

#define LED_PIN   4                             // ESP32 pin connected to LED's pin

// the setup routine runs once when you press reset:

void setup() {

  // initialize serial communication at 9600 bits per second:

  Serial.begin(115200);

  // declare LED pin to be an output and potentiometer as a input:

pinMode(POTENTIOMETER_PIN, INPUT);

  pinMode(LED_PIN, OUTPUT);

}
```

```
void loop() {

  int analogValue = analogRead(POTENTIOMETER_PIN);     //   reads the input on analog pin A0 (value between 0 and 4095)

  int brightness = map(analogValue, 0, 4095, 0, 255);   // scales it to brightness (value between 0 and 255)

    analogWrite(LED_PIN, brightness);                                  // sets the brightness LED that connects to  pin 26

  // print out the value

  Serial.print("Analog value = ");

  Serial.print(analogValue);

  Serial.print(" => brightness = ");

  Serial.println(brightness);

  delay(100);

}
```

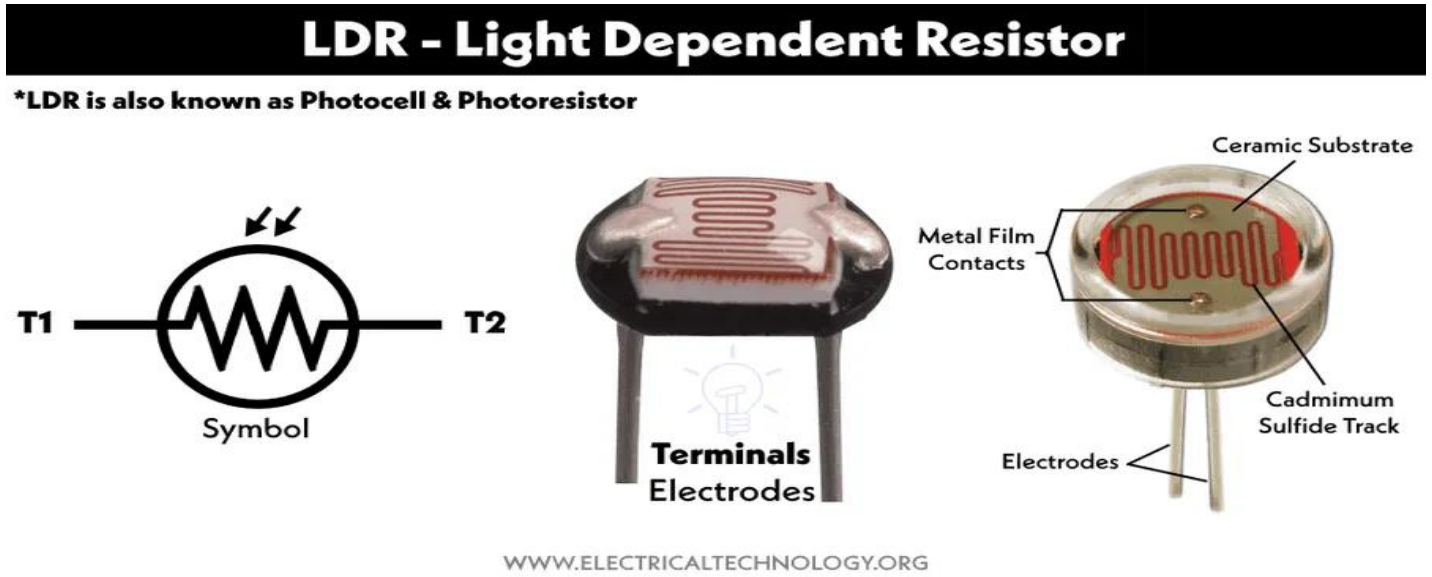

GND SIG VCC
ESP32
EN
Boot




```
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
Analog value = 1473 => brightness = 91
```

Smart lighting system using LDR and NodeMCU.

A Light Dependent Resistor (LDR), or photoresistor, is a passive electronic component whose resistance decreases as light intensity increases, based on the principle of photoconductivity. Commonly made from cadmium sulfide (CdS), they are used in light-sensitive circuits like automatic streetlights, alarm systems, and camera light meters. LDR is also known as a Light Sensor



Key Aspects of LDRs:

Working Principle: It works on the principle of photoconductivity. Whenever the light falls on its **photoconductive material**, it **absorbs its energy** and the electrons of that photoconductive material in the **valence band get excited and go to the conduction band and thus increasing the conductivity as per the increase in light intensity.**

Also, the energy in incident **light should be greater than the bandgap gap energy so** that the electrons from the valence band got excited and go to the conduction band.

In darkness, an LDR has a very high resistance (sometimes Megaohms), but as light strikes the surface, more electrons are released, causing the resistance to drop to a few hundred Ohms.

Structure: They consist of a zigzag pattern of photosensitive material, such as Cadmium Sulfide (CdS) or Cadmium Selenide (CdSe), usually housed in a transparent casing.

Arduino/ESP32 code to get the analog reading (`ldrVal`) and convert it to voltage:

$$V_{out} = \frac{ADC_{val}}{ADC_{resolution}} \times V_{supply}$$

(e.g., $\frac{1560}{4095} \times 3.3V \approx 1.25V$ if using an ESP32 with 3.3V supply and 12-bit ADC).

Estimated Threshold Voltages (3.3V Supply)

For 300 Lux: Approx 0.5V - 0.8V (Analog Reading: ~600 - 1000)

For 200 Lux: Approx 0.8V - 1.2V (Analog Reading: ~1000 - 1500)

- ldrRead = analogRead(ldr); //saving the read into the "ldrRead" variable.
- voltage = (ldrRead / 4095) * 3.3; //calculating the voltage.

- Use if (current_light_value < threshold) to detect **darkness** (simulating low light).
- Use if (current_light_value > threshold) to detect **brightness**.

1 ADC value → Voltage

ESP32 ADC is 12-bit:

$$V_{out} = \frac{ADC}{4095} \times V_{ref}$$

$$V_{out} = 5V \times \frac{R_{10k}}{R_{LDR} + R_{10k}}$$

Wokwi's "Photoresistor Sensor" module usually follows this logic by default:

Bright Light (High Lux): The sensor output voltage drops. analogRead() gives a low number (e.g., 100).

Darkness (Low Lux): The sensor output voltage rises. analogRead() gives a high number (e.g., 900).

If your circuit behaves like the one above (High Lux = Low Value),

To turn ON a light when it's DARK:

IF (CURRENT_VALUE > THRESHOLD)

(Example: If current is 900 and threshold is 500, it's dark enough to trigger).

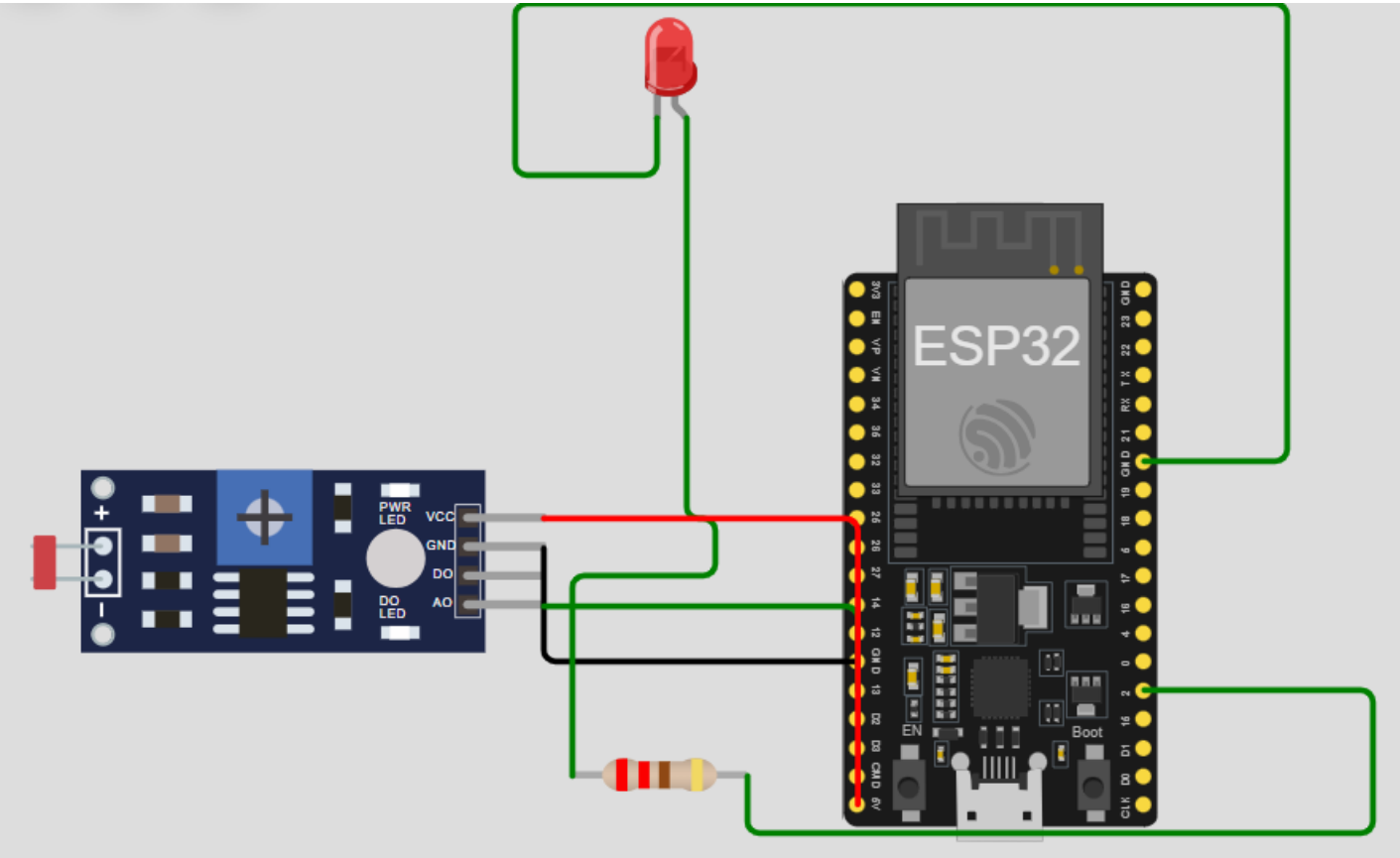
To turn ON a light when it's BRIGHT:

IF (CURRENT_VALUE < THRESHOLD)

(Example: If current is 100 and threshold is 500, it's bright enough to trigger).

```
int LDRPin = 13; // LDR connected to analog pin A0

int LEDPin = 2; // LED connected to digital pin 13

int threshold = 1560; // Threshold value for light level (adjust as needed) 200lux

void setup() {

  pinMode(LDRPin, INPUT); // Set LDR pin as input

  pinMode(LEDPin, OUTPUT); // Set LED pin as output

  Serial.begin(115200); // Initialize serial communication for debugging

}
```

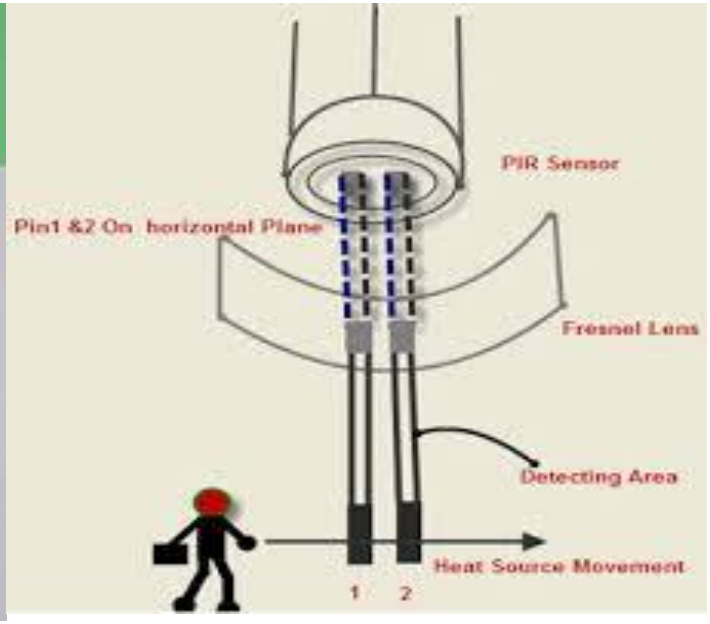
```
void loop() {
  int LDRValue = analogRead(LDRPin); // Read the value from the LDR
  Serial.println(LDRValue); // Print the LDR value to the Serial Monitor for debugging
  if (LDRValue > threshold) {
    // If the light level is above the threshold, turn on the LED
    digitalWrite(LEDPin, HIGH);
  } else {
    // If the light level is below the threshold, turn off the LED
    digitalWrite(LEDPin, LOW);
  }
```

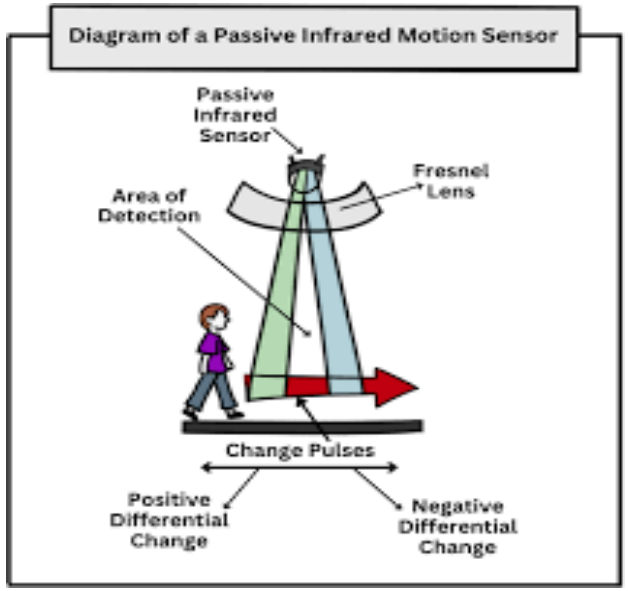


When it’s dark (ADC reading high), LED turns ON.When it’s bright (ADC reading low), LED turns OFF.

MOTION DETECTION USING PIR SENSOR AND ESP32

PIR (Passive Infrared) sensors detect motion by measuring rapid changes in infrared (IR) radiation emitted by warm objects, such as humans or animals, within their field of view. When a person moves across the sensor's range, the two pyroelectric sensors inside detect a differential change in IR energy—first one half, then the other—triggering a high voltage signa





Core Working Principle

IR Emission: Every object with a temperature above absolute zero emits infrared radiation. Humans and animals, being warm-blooded, emit significant IR energy in the 8–14 µm wavelength range.

Dual-Slot Detection: Inside a PIR module is a **pyroelectric sensor split into two balanced halves (slots**). When the sensor is **idle, both slots detect the same ambient IR level (from walls, furniture, etc**.), and their outputs cancel each other out.

Differential Signal: When a warm body moves across the sensor:

It first passes the first slot, causing a **positive differential change in voltage**.

As it moves to the second slot, it **causes a negative differential change.**

This "pulse" or difference in voltage is what the sensor interprets as motion

Key Components

Fresnel Lens: The plastic dome on the sensor is a Fresnel lens. It **condenses IR light from a wide area and focuses it onto the pyroelectric sensor.** It also splits the **detection area into multiple "zones" to help the sensor see movement more clearly.**

Pyroelectric Material: A special crystal (often in a metal can) that generates an **electrical charge when exposed to temperature changes.**

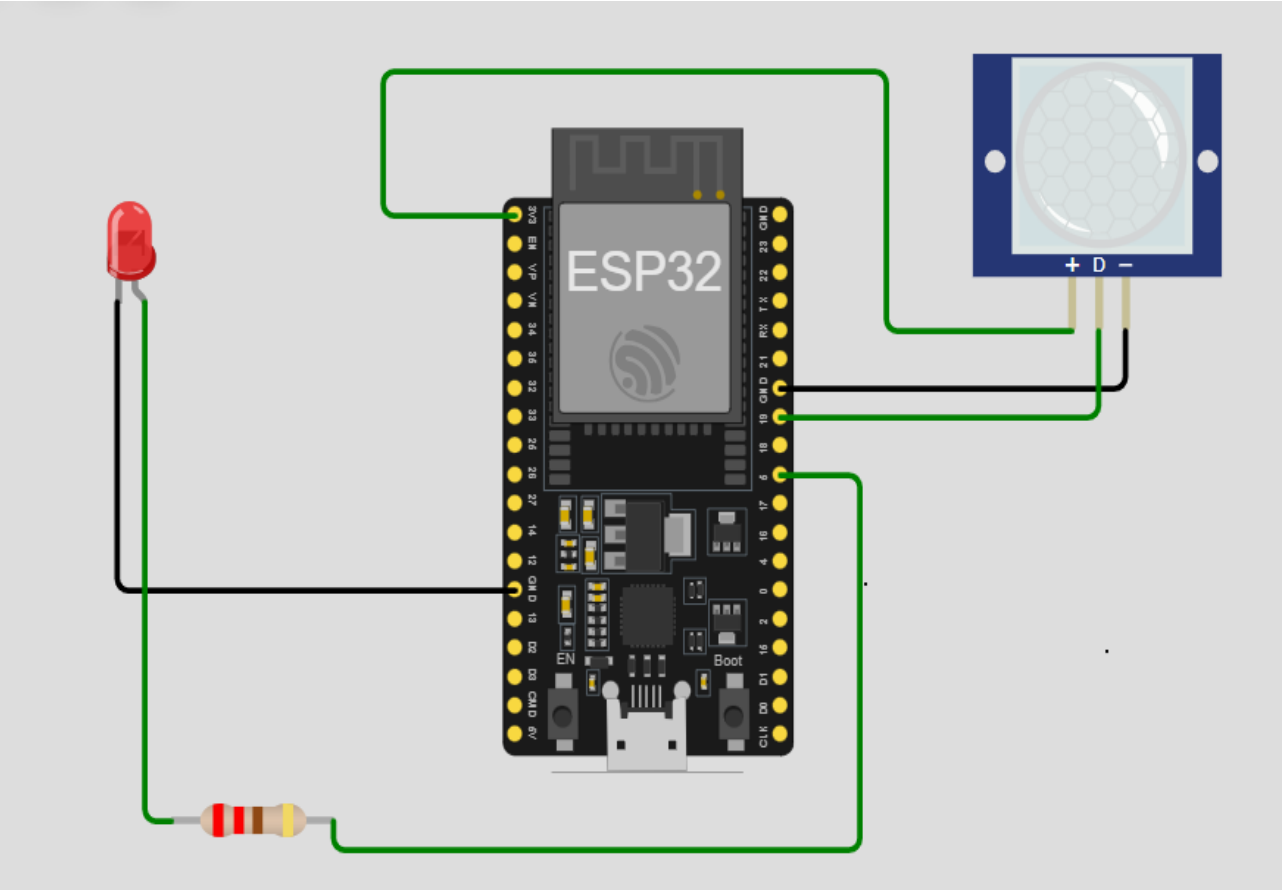
Adjustment Knobs (Potentiometers): Most modules, like the popular HC-SR501, feature two adjustment knobs:

Sensitivity: Controls the detection distance (typically 3–7 metres).

Time Delay: Sets how long the output signal stays "High" after motion is detected (from seconds to minutes)

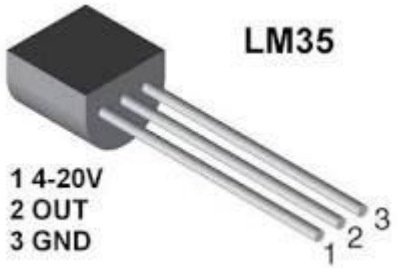
```
#define led 5
#define pir 19
int IP=0;  //#define IP 0;
void setup(){
  pinMode(led,OUTPUT);
  pinMode(pir,INPUT);
  Serial.begin(115200);
}
void loop(){
  IP=digitalRead(pir);
   delay(100);
```

```
if(IP==1){
  digitalWrite(led,HIGH);
   Serial.print(IP);
   Serial.println("=>Object is detected");
  delay(1000);
}
else{
  digitalWrite(led,LOW);
  Serial.print(IP);
  Serial.println("=>no Object  is detected");
  delay(1000);
  }
}
```



TEMPERATURE MEASUREMENT USING THERMISTOR-LM35/DHT11/DHT22 AND ESP32:

The **LM35** is a popular, precision **analog temperature sensor IC** that provides an output voltage linearly proportional to the temperature in degrees Celsius.The LM35 temperature sensor works **by converting temperature changes into a proportional analog voltage signal,** based on the physical principle that the **voltage across a diode-connected transistor changes with temperature**. This voltage is then amplified and calibrated internally to provide a linear output in degrees Celsius. **LM35**: The code reads the analog voltage from the ADC pin, converts it to **millivolts, and divides by 10 to get the Celsius temperature.**



```
#define LM35_PIN 34  // Analog pin where LM35 Vout is connected
void setup() {
  Serial.begin(115200);
  // Optional: Set ADC attenuation for better range (0-3.3V)
  analogSetAttenuation(ADC_11db);
}
void loop() {
  // Read the raw ADC value (0 to 4095 for 12-bit)
  int rawADC = analogRead(LM35_PIN);
  // Convert raw value to voltage (mV)
  // Formula: (Reading / 4095) * 3300mV
  float voltage = (rawADC / 4095.0) * 3300.0;
```

```
// Convert voltage to Celsius (10mV = 1°C)
  float tempC = voltage / 10.0;
  // Print results
  Serial.print("Raw ADC: ");
  Serial.print(rawADC);
  Serial.print(" | Voltage: ");
  Serial.print(voltage);
  Serial.print("mV | Temp: ");
  Serial.print(tempC);
  Serial.println("°C");

  delay(1000); // Wait 1 second between readings
}
```

The **DHT22** (also known as the **AM2302**) is a popular, low-cost digital sensor for simultaneously measuring both **temperature and humidity**.

You'll need to install the DHT sensor library in the Arduino IDE and use functions like **dht.readTemperature()** to get the data.

Install Libraries: Use the Arduino IDE Library Manager to install the **"DHT sensor library" by Adafruit**.

DHT22 pins	
1	VCC
2	DATA
3	NC
4	GND

1. Humidity Sensing (Capacitive Principle)

The sensor uses a **capacitive humidity sensing element** to measure relative humidity.

Structure: It consists of two electrodes with a **moisture-holding substrate (often a polymer or salt) acting as a dielectric between them.**

Mechanism: As humidity in the **air changes, the substrate absorbs or releases water vapor. This** changes the **dielectric constant, which in turn alters the capacitance between the electrodes.**

Processing: An internal circuit measures this change in **capacitance and converts it into a digital humidity** value.

2. Temperature Sensing (NTC Thermistor)

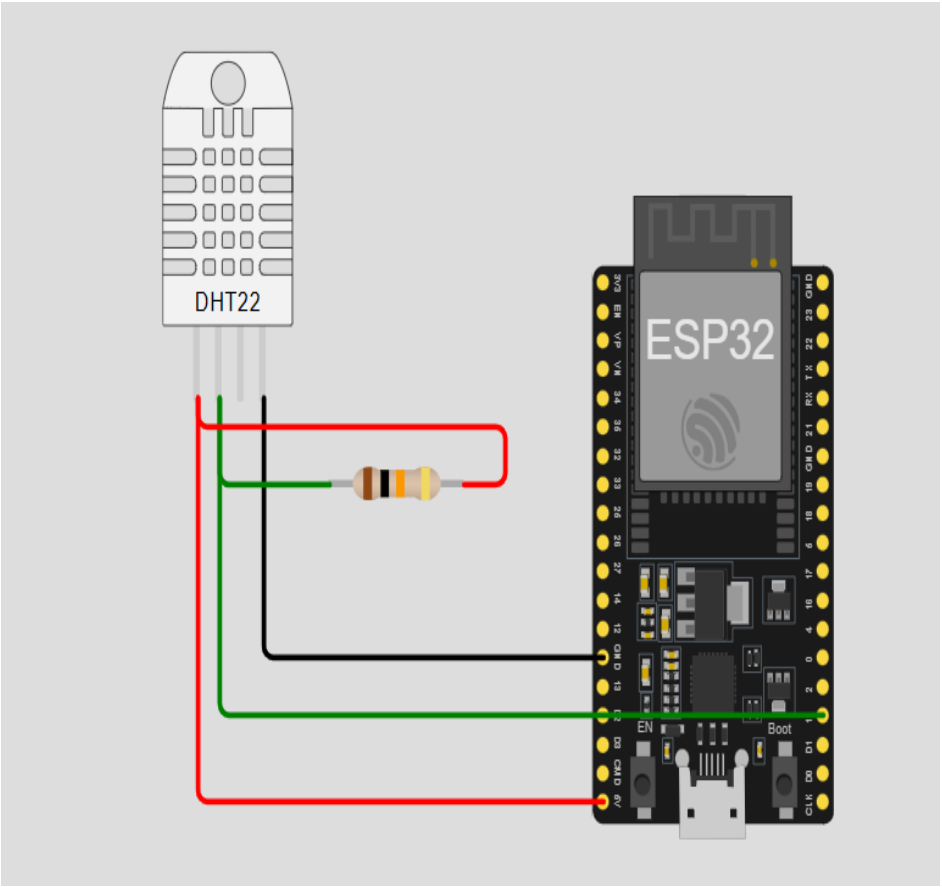
For temperature, the DHT22 typically uses an **NTC (Negative Temperature Coefficient**) thermistor.

Mechanism: A thermistor is a variable resistor whose **resistance decreases as the temperature increases.**

Sensing: Small changes in environmental temperature lead to large, measurable changes in the thermistor's resistance.

Bandgap Alternative: Some high-end variants utilize a **silicon bandgap temperature sensor**, which produces **a voltage proportional to absolute temperature for even higher precision**

```
#include "DHTesp.h"
const int DHT_PIN = 15;
DHTesp dhtSensor;
void setup() {
  Serial.begin(115200);
  dhtSensor.setup(DHT_PIN, DHTesp::DHT22);
}
void loop() {
  TempAndHumidity  data = dhtSensor.getTempAndHumidity();
  Serial.println("Temp: " + String(data.temperature, 2) + "°C");
  Serial.println("Humidity: " + String(data.humidity, 1) + "%");
  Serial.println("---");
  delay(2000); // Wait for a new reading from the sensor (DHT22 has ~0.5Hz sample rate)
}
```



dhtSensor.setup(DHT_PIN, DHTesp::DHT22);

dhtSensor→ Object of the DHTesp class (your sensor handler)

setup(...)→ Tells the library which pin and which DHT sensor typeLike:he GPIO pin number where the **DATA** pin of the DHT sensor is connected

If we are using DHT11

Then dhtSensor.setup(DHT_PIN, DHTesp::DHT11);

TempAndHumidity data = dhtSensor.getTempAndHumidity();

TempAndHumidity

•A **structure (struct)** defined inside the **DHTesp** library

•It contains **two float values**:

- temperature → in **°C**
- humidity → in **%**

```
struct TempAndHumidity {
float temperature;
float humidity;
};
```

dhtSensor.getTempAndHumidity()

Communicates with the **DHT22 sensor**

Reads raw digital data Converts it into **temperature and humidity**

Returns both values together as a TempAndHumidity object

data

A variable that **stores the returned readings**

You access values like:

data.temperature

data.humidity